

DXGI GPU INFO

A Plain-English Guide to Querying GPU Memory
on Windows Using Python [ctypes](#)

No extra libraries

Windows only

AMD · NVIDIA · Intel

Pure Python

What you will learn

- ✓ What COM and DXGI are and why we need them
- ✓ How vtables let Python call Windows C++ interfaces
- ✓ Every line of the final script, explained simply
- ✓ How to reuse this pattern for your own projects

1 Background — What Are We Even Doing?

Windows keeps track of every graphics card in your PC. To ask Windows "how much VRAM does my GPU have?", we have to speak its language — a system called **COM** (Component Object Model) exposed through an API called **DXGI** (DirectX Graphics Infrastructure). Python's built-in `ctypes` library lets us call these Windows interfaces directly, with zero extra installs.

Term	Plain English
COM	A Windows system that lets programs talk to each other using agreed-upon rules (interfaces). Think of it like a universal plug socket.
DXGI	Microsoft's graphics layer. It knows about all your GPUs and can report their memory, names, and capabilities.
ctypes	A Python module that lets you call raw C/C++ Windows functions as if they were normal Python functions.
HRESULT	A number Windows returns from every COM call. 0x0 means success. Anything else is an error code.
vtable	A lookup table of function pointers inside a COM object. We find the right slot number to call the right method.
IID	A unique ID (GUID) that identifies a specific COM interface, like IDXGIFactory1 or IDXGIAdapter3.

Analogy: Imagine DXGI is a hotel front desk. COM is the intercom system you use to call them. The vtable is the hotel's internal phone directory — each room (method) has a slot number. We dial the right slot to get the information we want.

2 The Complete Script

Here is the entire working script. The sections that follow break down every piece.

```
import ctypes

DXGI_MEMORY_SEGMENT_GROUP_LOCAL = 0
VENDOR_AMD = 0x1002
VENDOR_NVIDIA = 0x10DE
VENDOR_INTEL = 0x8086

class DXGI_QUERY_VIDEO_MEMORY_INFO(ctypes.Structure):
    _fields_ = [
        ("Budget", ctypes.c_uint64),
        ("CurrentUsage", ctypes.c_uint64),
        ("AvailableForReservation", ctypes.c_uint64),
        ("CurrentReservation", ctypes.c_uint64),
    ]

class DXGI_ADAPTER_DESC(ctypes.Structure):
    _fields_ = [
        ("Description", ctypes.c_wchar * 128),
        ("VendorId", ctypes.c_uint),
        ("DedicatedVideoMemory", ctypes.c_size_t),
        # ... (other fields omitted for brevity)
    ]

IID_IDXGIFactory1 = (ctypes.c_byte * 16)(
    0x78, 0xae, 0x0a, 0x77, 0x6f, 0xf2, 0xba, 0x4d,
    0xa8, 0x29, 0x25, 0x3c, 0x83, 0xd1, 0xb3, 0x87
)

IID_IDXGIAdapter3 = (ctypes.c_byte * 16)(
    0xA4, 0x67, 0x59, 0x64, 0x92, 0x13, 0x10, 0x43,
    0xA7, 0x98, 0x80, 0x53, 0xCE, 0x3E, 0x93, 0xFD
)

def _vtable_fn(obj, slot, restype, *argtypes):
    vtable = ctypes.cast(obj, ctypes.POINTER(ctypes.c_void_p))
    fn_ptr = ctypes.cast(vtable[0], ctypes.POINTER(ctypes.c_void_p))[slot]
    return ctypes.cast(fn_ptr, ctypes.CFUNCTYPE(restype, ctypes.c_void_p, *argtypes))

def _release(obj):
```

```

        _vtable_fn(obj, 2, ctypes.c_ulong)(obj)

def _query_interface(obj, iid, out):
    fn = _vtable_fn(obj, 0, ctypes.c_long,
                    ctypes.POINTER(ctypes.c_byte * 16),
                    ctypes.POINTER(ctypes.c_void_p))
    return fn(obj, iid, out)

def get_dxgi_gpu_info() -> dict:
    ctypes.windll.ole32.CoInitialize(None) # start COM
    factory = ctypes.c_void_p()
    if ctypes.windll.dxgi.CreateDXGIFactory1(
        ctypes.byref(IID_IDXGIFactory1), ctypes.byref(factory)) != 0:
        ctypes.windll.ole32.CoUninitialize()
        return {}
    enum_adapters = _vtable_fn(factory, 12, ctypes.c_long,
                                ctypes.c_uint, ctypes.POINTER(ctypes.c_void_p))

    gpus = {}
    index = 0
    while True:
        adapter1 = ctypes.c_void_p()
        if ctypes.c_uint32(enum_adapters(factory, index,
                                         ctypes.byref(adapter1))).value != 0: break
        adapter3 = ctypes.c_void_p()
        qi_hr = _query_interface(adapter1,
                                ctypes.byref(IID_IDXGIAdapter3), ctypes.byref(adapter3))
        _release(adapter1)
        if ctypes.c_uint32(qi_hr).value != 0:
            index += 1; continue
        desc = DXGI_ADAPTER_DESC()
        mem_info = DXGI_QUERY_VIDEO_MEMORY_INFO()
        _vtable_fn(adapter3, 8, ctypes.c_long,
                    ctypes.POINTER(DXGI_ADAPTER_DESC))(adapter3, ctypes.byref(desc))
        _vtable_fn(adapter3, 14, ctypes.c_long,
                    ctypes.c_uint, ctypes.c_uint,
                    ctypes.POINTER(DXGI_QUERY_VIDEO_MEMORY_INFO))(
            adapter3, 0, 0, ctypes.byref(mem_info))
        total = desc.DedicatedVideoMemory // (1024 ** 3)

        free = (mem_info.Budget - mem_info.CurrentUsage) // (1024 ** 3)
        if total > 0 and desc.VendorId in (0x1002, 0x10DE, 0x8086)
            and free <= total:
            gpus[index] = {'name': desc.Description,
                           'totalMem': total, 'freeMem': free}
        _release(adapter3)
        index += 1
    _release(factory)
    ctypes.windll.ole32.CoUninitialize()
    return gpus

```

3 Step 1 — Defining C Structures

Windows functions pass data around using **C structs** — fixed memory layouts where each field lives at a known byte offset. We mirror those layouts in Python using `ctypes.Structure` so Python and Windows agree on exactly where each piece of data lives in memory.

DXGI_QUERY_VIDEO_MEMORY_INFO

```
class DXGI_QUERY_VIDEO_MEMORY_INFO(ctypes.Structure):
    _fields_ = [
        ("Budget",          ctypes.c_uint64), # total VRAM OS allows us
        ("CurrentUsage",     ctypes.c_uint64), # VRAM in use right now
        ("AvailableForReservation", ctypes.c_uint64), # we can reserve this much
        ("CurrentReservation", ctypes.c_uint64), # we have reserved this much
    ]
```

Budget is the key field. Think of it as your weekly allowance — the OS decides how much VRAM your process is allowed to use. **CurrentUsage** is how much you've already spent. Free memory = Budget – CurrentUsage.

DXGI_ADAPTER_DESC

```
class DXGI_ADAPTER_DESC(ctypes.Structure):
    _fields_ = [
        ("Description",      ctypes.c_wchar * 128), # GPU name string
        ("VendorId",         ctypes.c_uint),        # maker: AMD/NVIDIA/Intel
        ("DeviceId",         ctypes.c_uint),        # specific model number
        ("SubSysId",         ctypes.c_uint),        # sub-system ID
        ("Revision",         ctypes.c_uint),        # hardware revision
        ("DedicatedVideoMemory", ctypes.c_size_t),  # total VRAM in bytes
        ("DedicatedSystemMemory", ctypes.c_size_t), # system RAM used as VRAM
        ("SharedSystemMemory", ctypes.c_size_t),    # shared RAM available
        ("AdapterLuid",       ctypes.c_byte * 8),   # unique adapter ID
    ]
```

DedicatedVideoMemory is the physical VRAM soldered onto your GPU. This is the 'true' total — not shared or borrowed system memory. **VendorId** lets us filter out iGPUs and software renderers.

4 Step 2 — Interface IDs (GUIDs)

Every COM interface has a globally unique 16-byte ID called a **GUID** (or **IID** when identifying an interface). We use these to ask Windows "give me THIS specific interface". The bytes look cryptic but they are just fixed constants from Microsoft's documentation — you never have to calculate them.

```
# IDXGIFactory1 {770AAE78-F26F-4DBA-A829-253C83D1B387}
IID_IDXGIFactory1 = (ctypes.c_byte * 16)(
    0x78, 0xae, 0x0a, 0x77, # first chunk, little-endian
    0x6f, 0xf2, # second chunk, little-endian
    0xba, 0x4d, # third chunk, little-endian
    0xa8, 0x29, # remaining bytes, big-endian
    0x25, 0x3c, 0x83, 0xd1, 0xb3, 0x87
)

# IDXGIAdapter3 {645967A4-1392-4310-A798-8053CE3E93FD}
IID_IDXGIAdapter3 = (ctypes.c_byte * 16)(
    0xA4, 0x67, 0x59, 0x64,
    0x92, 0x13,
    0x10, 0x43,
    0xA7, 0x98,
    0x80, 0x53, 0xCE, 0x3E, 0x93, 0xFD
)
```

Endianness gotcha: GUIDs are written as {XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX}. The first three groups are stored in little-endian order (bytes reversed). The last two groups are big-endian (bytes as-is). This is a well-known COM convention and the source of many subtle bugs.

Term	Plain English
IDXGIFactory1	The 'factory' interface — creates and enumerates adapters
IDXGIAdapter1	A basic adapter handle returned by EnumAdapters1
IDXGIAdapter3	An upgraded adapter handle that adds QueryVideoMemoryInfo

5 Step 3 — The vtable Helper Function

COM interfaces are C++ objects. In C++, every object has a hidden pointer called a **vtable** (virtual function table) — it's a list of function pointers, one for each method. To call a method, we look up its slot number in that table and call the function at that address.

```
def _vtable_fn(obj, slot, restype, *argtypes):
    # Step 1: treat the COM object pointer as a pointer-to-pointer.
    #   The first pointer points to the vtable.
    vtable = ctypes.cast(obj, ctypes.POINTER(ctypes.c_void_p))

    # Step 2: dereference vtable[0] to get the vtable address,
    #   then index into it at [slot] to get the function pointer.
    fn_ptr = ctypes.cast(vtable[0], ctypes.POINTER(ctypes.c_void_p))[slot]

    # Step 3: cast that raw address into a typed callable so ctypes
    #   knows how to marshal arguments and the return value.
    return ctypes.cast(fn_ptr,
        ctypes.CFUNCTYPE(restype, ctypes.c_void_p, *argtypes))
```

Memory diagram: COM object → [vtable ptr] → [fn0, fn1, fn2, fn3, ...] ↑ ↑ slot 0 (QueryInterface) slot 2 (Release) We read vtable[0] to get the address of the vtable, then index into that array to get the specific function we want to call.

Slot numbers used in this script

Term	Plain English
slot 0	QueryInterface — ask the object to give us a different interface version
slot 2	Release — tell COM we're done with this object (memory management)
slot 8	GetDesc — get the adapter description (name, VRAM total, vendor ID)
slot 12	EnumAdapters1 — list all GPUs (called on the factory object)
slot 14	QueryVideoMemoryInfo — get budget and current usage (IDXGIAdapter3 only)

6 Step 4 — Small Helper Functions

`_release(obj)`

```
def _release(obj):  
    _vtable_fn(obj, 2, ctypes.c_ulong)(obj)
```

COM objects must be manually released when you're done with them. Slot 2 is always `Release` on every COM interface (inherited from `IUnknown`). Forgetting to call this leaks memory.

`_query_interface(obj, iid, out)`

```
def _query_interface(obj, iid, out):  
    fn = _vtable_fn(obj, 0, ctypes.c_long,  
                    ctypes.POINTER(ctypes.c_byte * 16),  
                    ctypes.POINTER(ctypes.c_void_p))  
    return fn(obj, iid, out)
```

Slot 0 is always `QueryInterface`. It lets you ask a COM object: *"Can you also act as this other interface?"* We use this to upgrade an `IDXGIAdapter1` (basic) into an `IDXGIAdapter3` (has `QueryVideoMemoryInfo`). The IID tells it which interface we want. The result comes back via the `out` pointer.

Why upgrade from Adapter1 to Adapter3? `EnumAdapters1` returns `IDXGIAdapter1` objects. But `QueryVideoMemoryInfo` only exists on `IDXGIAdapter3` (added in DXGI 1.4). `QueryInterface` is the COM way to ask for the upgraded version of the same object.

7 Step 5 — The Main Function, Line by Line

■ Initialize COM

```
ctypes.windll.ole32.CoInitialize(None)
```

COM must be initialized before any COM calls are made. Without this, methods like `QueryVideoMemoryInfo` silently return zeros. `CoUninitialize()` is called at the end to clean up.

■ Create the DXGI Factory

```
factory = ctypes.c_void_p()
hr = ctypes.windll.dxgi.CreatedDXGIFactory1(
    ctypes.byref(IID_IDXGIFactory1), ctypes.byref(factory))
if hr != 0:
    ctypes.windll.ole32.CoUninitialize()
    return {}
```

We call `CreatedDXGIFactory1` from `dxgi.dll`. It gives us a factory object — think of it as the directory of all GPUs. We pass the `IID_IDXGIFactory1` so Windows knows what kind of factory interface to give back. If it fails (`hr != 0`) we bail out early.

■ Get the EnumAdapters1 function

```
enum_adapters = _vtable_fn(factory, 12, ctypes.c_long,
                             ctypes.c_uint,
                             ctypes.POINTER(ctypes.c_void_p))
```

We look up slot 12 on the factory — that's `EnumAdapters1`. We describe its signature: returns a `c_long` (HRESULT), takes a `c_uint` (index) and a pointer for the output adapter.

■ Loop through all adapters

```
index = 0
while True:
    adapter1 = ctypes.c_void_p()
    hr = enum_adapters(factory, index, ctypes.byref(adapter1))
    if ctypes.c_uint32(hr).value != 0:
        break    # no more adapters
```

We keep asking for adapter 0, 1, 2... until Windows returns a non-zero HRESULT meaning 'no more adapters'. We cast `hr` to `uint32` first because Python can interpret the same bit pattern as a negative signed integer — `uint32` gives us the true Windows error code.

■ Upgrade to IDXGIAdapter3

```

adapter3 = ctypes.c_void_p()
qi_hr = _query_interface(adapter1,
    ctypes.byref(IID_IDXGIAdapter3),
    ctypes.byref(adapter3))
_release(adapter1) # always release what we no longer need
if ctypes.c_uint32(qi_hr).value != 0:
    index += 1; continue

```

We try to upgrade `adapter1` to `adapter3` via `QueryInterface`. We immediately release `adapter1` — we only needed it to get `adapter3`. If the upgrade fails (e.g. very old driver), we skip this adapter and move on.

■ Read GPU description and memory info

```

desc      = DXGI_ADAPTER_DESC()
mem_info  = DXGI_QUERY_VIDEO_MEMORY_INFO()
# slot 8 = GetDesc
_vtable_fn(adapter3, 8, ctypes.c_long,
    ctypes.POINTER(DXGI_ADAPTER_DESC))
(adapter3, ctypes.byref(desc))
# slot 14 = QueryVideoMemoryInfo
_vtable_fn(adapter3, 14, ctypes.c_long,
    ctypes.c_uint, ctypes.c_uint,
    ctypes.POINTER(DXGI_QUERY_VIDEO_MEMORY_INFO))
(adapter3, 0, 0, ctypes.byref(mem_info))

```

We allocate empty C structs, then pass them by reference (byref) to the Windows functions, which fill them in. Slot 8 fills in the name and total VRAM. Slot 14 fills in the budget and current usage. The two zeros are `NodeIndex` (0 = first GPU node) and `MemorySegmentGroup` (0 = local/dedicated VRAM).

■ Filter and store results

```
total = desc.DedicatedVideoMemory // (1024 ** 3)
free  = (mem_info.Budget - mem_info.CurrentUsage) // (1024 ** 3)
if total > 0
    and desc.VendorId in (0x1002, 0x10DE, 0x8086)
    and free <= total:
        gpus[index] = {
            'name':      desc.Description,
            'totalMem': total,
            'freeMem':   free,
        }
    _release(adapter3)
    index += 1
```

We convert bytes to GB (divide by 1024 three times). The three filter conditions drop: software renderers with no real VRAM (total == 0), unknown vendors, and iGPUs whose 'budget' is inflated shared system RAM (free > total is a dead giveaway). Always release adapter3 at the end of each loop iteration.

8 How to Reuse This Pattern

The vtable pattern here works for *any* COM interface on Windows. Here is the recipe to follow for your own project:

1	Find the interface in Microsoft Docs Search 'IDXGIWhatever site:learn.microsoft.com'. The docs list every method in order — that order is your vtable slot index (starting from 0, including inherited methods).
2	Count inherited slots COM interfaces inherit from each other. Count all methods from IUnknown (3) + IDXGIObject (4) + IDXGIAdapter (3) + ... then add the new methods. The first new method starts at the sum.
3	Mirror the C structs Any struct passed to/from the function needs a matching ctypes.Structure subclass with <code>_fields_</code> in the exact same order and types as the C definition.
4	Get the IID bytes right Copy the GUID from docs. The first 3 groups are little-endian (reverse each group's bytes). The last 2 groups are big-endian (copy as-is).
5	Always call CoInitialize first Without CoInitialize, many COM methods silently fail or return zeroed structs. Always uninitialized at the end too.
6	Always Release what you're done with Every COM object is reference-counted. Call Release (slot 2) when you're finished with each object or you will leak memory.

Finding slot numbers reliably: The safest way is to look at the C++ header file directly. Search GitHub for the interface name in `dxgi.h`, `dxgi1_2.h`, `dxgi1_4.h` etc. Count the virtual methods in order — each one is a slot.

9 Quick Reference

Vendor IDs

Term	Plain English
<code>0x1002</code>	AMD — includes all Radeon discrete and integrated GPUs
<code>0x10DE</code>	NVIDIA — all GeForce and Quadro GPUs
<code>0x8086</code>	Intel — integrated graphics (UHD, Iris, Arc)

Common HRESULT codes

Term	Plain English
<code>0x00000000</code>	S_OK — success
<code>0x80004002</code>	E_NOINTERFACE — wrong IID, interface not supported
<code>0x80004001</code>	E_NOTIMPL — wrong vtable slot, method not implemented
<code>0x887A0002</code>	DXGI_ERROR_NOT_FOUND — no more adapters to enumerate

ctypes type cheat-sheet

Term	Plain English
<code>c_uint32</code>	Unsigned 32-bit integer — use for HRESULT comparisons
<code>c_uint64</code>	Unsigned 64-bit integer — VRAM sizes in bytes
<code>c_size_t</code>	Pointer-sized integer — correct for DedicatedVideoMemory
<code>c_void_p</code>	Generic pointer — used for COM object handles
<code>c_wchar*128</code>	Fixed-length wide string — GPU name from DXGI_ADAPTER_DESC
<code>byref(x)</code>	Pass x by pointer — lets C code write back into x
<code>CFUNCTYPE</code>	Declares a C function signature Python can call safely

Tip — refreshing free memory: To get a live freeMem reading, just call `get_dxgi_gpu_info()` again. The Budget and CurrentUsage values are always current as of the moment the call is made. For real-time monitoring, call it on a timer in a background thread.